

Examen réparti 1 DataCloud

Master 2 SAR

Jonathan Lejeune

22 Novembre 2022

2 heures – **Tout document papier interdit**
Tout appareil électronique interdit

- Le barème est sur $21\frac{1}{2}$ points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Lors de la correction il sera tenu compte de la simplicité et de la lisibilité de vos réponses
- S'abstenir de répondre est préférable à répondre une absurdité
- Vous trouverez en annexe de la documentation sur l'API de Spark (RDD + DStream)

Exercice 1 – Questions de cours (8 points)

Question 1 1 point

Qu'est ce que le map-reduce ?

Question 2 1 point

Dans un traitement respectant le map-reduce, à quoi sert un partitionneur ? Quelle est la politique par défaut et pourquoi est-elle satisfaisante dans la plupart des cas ?

Question 3 1 point

Dans le HDFS, quel service est rendu par le SecondaryNameNode ? Que se passerait-il si ce service n'était pas déployé ?

Question 4 1 point

Dans YARN quel(s) mécanisme(s) empêche(nt) une application d'utiliser des conteneurs qui ne lui appartiennent pas.

Question 5 1 point

Soit les deux lignes scala suivantes :

```
1 val x = List(3, "bonjour", true)
2 val y = (3, "bonjour", true)
```

Quels sont les types inférés par le compilateur de x et y ? **Justifier.**

Question 6 1 point

Soit le code scala suivant :

```
1 final class Point(var x:Int, var y:Int){
2     override def toString="("+x+", "+y+")"
3 }
```

Sans modifier cette classe, donner le code nécessaire pour que les lignes suivantes puisse compiler et afficher (3,6)

```
1 val p = new Point(2,5)
2 p.move(1,1)
3 println(p)
```

Question 7 1 point

Sur des RDD, donnez un code à fonctionnalité équivalente de la méthode `reduceByKey(f)` en utilisant un `groupByKey`? Pourquoi est-il préférable d'utiliser `reduceByKey`?

Question 8 1 point

Dans une application Spark Streaming où le `StreamingContext` est instancié à une période égale à x , comment faire pour avoir dans la même application des Dstream de période x et des DStream de période y ($y \neq x$)? Quelle(s) contrainte(s) doit-on respecter sur la valeur de y ?

Exercice 2 – Kmeans (10 points)

Le k-means est une technique de partitionnement qui permet de classifier en k groupes un ensemble de points. Les points peuvent être multidimensionnels mais dans le cadre de cet exercice, nous allons nous limiter à une dimension 2 permettant ainsi de considérer des points dans le plan. Le principe est d'abord de prendre à l'initialisation un échantillon aléatoire de k points parmi les points initiaux et ensuite tendre vers k points qui minimisent la distance entre les différents points initiaux. L'algorithme se termine lorsque l'on atteint un point fixe sur le calcul. La figure 1 décrit un exemple d'exécution avec $k = 4$.

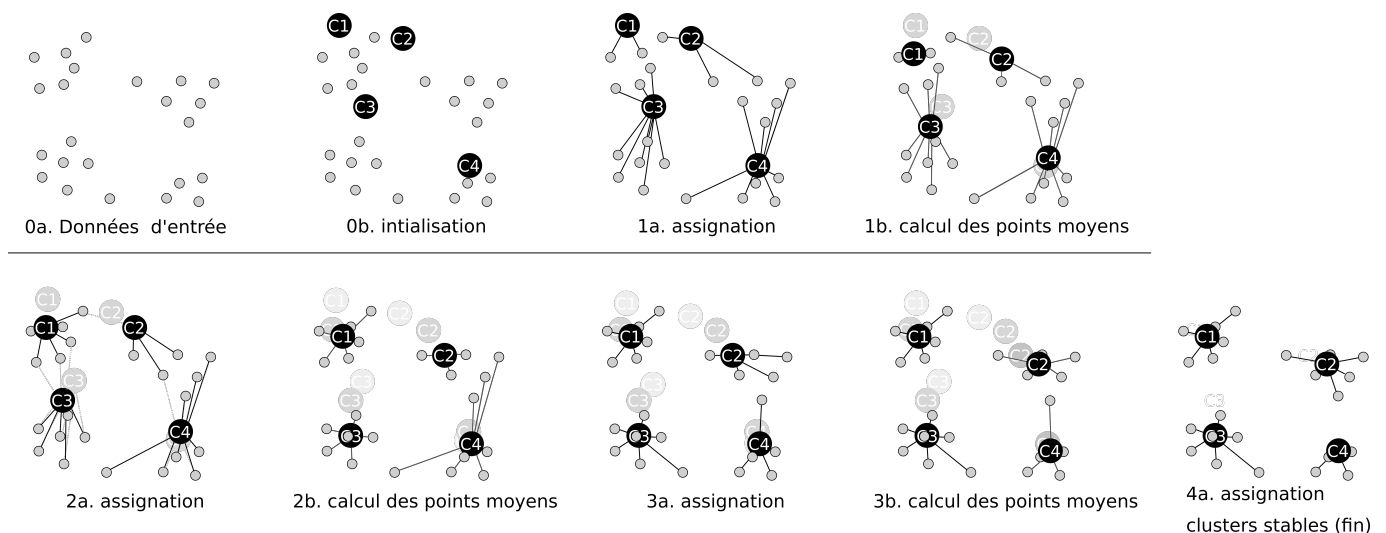


FIGURE 1 – Exemple d'exécution d'un algorithme de k-means (source : Wikipédia)

Soit la méthode `kmeans` ci-dessous qui implante l'algorithme du k-means avec les RDDs de Spark. Elle prend en paramètre l'ensemble des données d'entrée, le nombre de groupes à calculer et une valeur de distance minimale permettant de paramétrer la fin de l'algorithme.

```

1 def kmeans(data:RDD[PairDouble], k:Int, dist:Double)={
2   var kc = data.takeSample(withReplacement = false, k, seed=42)
3   var d = 0.0
4   do {
5     val a = data.map (p => (p.closest(kc), (p, 1)))
6     val s = a.reduceByKey( (a,b)=> (a._1 + b._1, a._2+b._2),numPartitions=k)
7     val nkc = s.map( c=> c._2._1 * (1.0 / c._2._2)).collect()
8     val tmp = kc.zip(nkc).map(c=>c._1.distance(c._2))
9     d=tmp.reduce(_+_ )
10    kc=nkc
11  }while( d > dist)
12
13  data.map(p => {
14    val group = p.closest(kc)
15    (p,(group, kc(group)))
16  })
17 }

```

La méthode `takeSample` (ligne 2) est définie dans la classe `RDD` et se comporte comme la méthode `take` sauf qu'elle renvoie k éléments aléatoires. Son implantation est basée sur un appel à `count` et à `collect` sur le `RDD` appelant.

L'entête de la classe `PairDouble` est définie ainsi : `case class PairDouble(x:Double, y:Double)` et comporte a minima la méthode `closest` qui pour un tableau de `PairDouble` donné, renvoie l'indice de l'élément du tableau le plus proche de l'instance en termes de distance euclidienne (le code de cette méthode n'est pas demandé).

Question 1 $\frac{1}{2}$ point

Rappeler la différence entre une `case class` et une `class`.

Question 2 1 point

Quelles seraient les conséquences sur la distribution du calcul si le paramètre `numPartitions` de la ligne 5 était différent de k (prendre le cas $> k$ et $< k$) ?

Question 3 1 point

Donner la signature des méthodes à ajouter dans la classe `PairDouble` pour que la fonction `kmeans` compile (leur corps n'est pas demandé).

Question 4 1 point

Quel sont les types des variables `kc`, `a`, `s`, `nkc` et `tmp` inférés par le compilateur ?

Question 5 $\frac{1}{2}$ point

Quel est le type de retour de la fonction `kmeans` ?

Question 6 1 point

Quels sont les morceaux de codes qui ne sont pas exécutés par le driver ?

Question 7 1 point

Après avoir rappelé le danger que peut engendrer un appel à `collect`, expliquer pourquoi celui de la ligne 7 n'est pas dangereux.

On suppose que les données d'entrée sont stockés sur HDFS dans des fichiers texte. Chaque ligne de ces fichiers représente un point et contient deux nombres à virgule séparés par un espace : le premier pour la valeur de x , le deuxième pour la valeur de y .

Question 8 $1\frac{1}{2}$ points

Dans le package `datacloud.exam`, écrire le code d'une application Spark nommée `Kmeans` qui prend au moins 4 arguments sur sa ligne de commande :

- la valeur de `k` (pour le deuxième argument de la fonction `kmeans`)
- la distance minimale (pour le troisième argument de `kmeans`)
- un chemin HDFS de sortie
- le reste des arguments étant une liste de chemins HDFS de fichiers texte d'entrée

L'application produit dans le chemin HDFS de sortie le résultat de la fonction `kmeans` qui a été paramétrée par un RDD qui contient les points stockés dans l'ensemble des fichiers texte d'entrée.

Après avoir bien démarré les services d'Hadoop avec son paramétrage par défaut, nous exécutons la commande suivante

```
spark-submit --master yarn --class datacloud.exam.Kmeans 3 0.01 /out /foo/f1 /foo/f2 /bar/g
```

Question 9 2 points

En supposant que :

- la méthode `kmeans` ait besoin de 2 tours de boucle pour atteindre le point fixe,
- que les fichiers `/foo/f1 /foo/f2 /bar/g` font respectivement 90Mo, 150Mo et 120Mo

Combien de jobs spark seront exécutés par ce programme ? Pour chacun de ces jobs donner l'instruction qui les déclenche et leur graphe de tâches ?

Question 10 $\frac{1}{2}$ point

Parmi les RDD manipulés par la fonction `kmeans` lesquels serait judicieux d'y appliquer la méthode `cache()` ? **Justifier.**

Exercice 3 – Usagers des transports (3,5 points)

La RATP, société de transport, gère un ensemble de stations de métro. Pour accéder au service, chaque usager peut soit utiliser un ticket soit une carte d'abonnement. À chaque fois qu'un usager entre dans le métro, un log textuel est produit par la borne. Ce log contient une date sous le format `JJ_MM_AAAA_hh_mm_ss`, le numéro de la carte d'abonnement ou du ticket de transport (un identifiant commençant par un `t`) et le nom de la station. Chaque information est séparée par un caractère d'espace. Par exemple le log `04_10_2022_13_22_34 1273453673 Place_Monge` signifie que la carte `1273453673` a badgé à la station Place Monge le 4 octobre 2022 à 13h 22min 24sec. Chaque ligne de log produite est envoyée vers un système d'agrégation de logs qui a la capacité de les re-transmettre ensuite en temps réel. On supposera que la source d'émission du système d'agrégation est la machine `stat.ratp.fr` sur le port 7815 et que l'on accédera à ce flux de log via une socket TCP.

Question 1 $1\frac{1}{2}$ points

On souhaite connaître une estimation du nombre d'usagers actuellement présents dans le métro. Puisque l'on connaît uniquement la date d'entrée de chaque usager (mais pas sa date de sortie) on suppose qu'un usager reste en moyenne 45 minutes dans le métro. Écrire un programme Spark qui affiche toutes les 30 secondes cette estimation.

Bonus : Vous ferez en sorte d'utiliser les primitives les plus adéquates afin de maximiser les performances.

Question 2 2 points

Écrire un programme Spark qui calcule en temps réel la proportion de tickets et de cartes d'abonnement qui ont badgé par station **depuis le début de l'acquisition** en pourcentage. À tout instant t la somme des deux valeurs doit donc être égale à 100%

API for SparkContext	Meaning
new SparkContext (config: SparkConf)	Create a SparkContext that loads settings from a Spark Config object describing the application configuration. Any settings in this config overrides the default configs as well as system properties.
new SparkContext (master: String, appName: String, conf: SparkConf)	Alternative constructor that allows setting common Spark properties directly, : Cluster URL to connect to (e.g. mesos://host:port, spark://host:port, local[4]), a name for your application, to display on the cluster web UI and a SparkConf object specifying other Spark parameters
textFile (path: String, minPartitions: Int = defaultMinPartitions): RDD[String]	Read a text file from HDFS, a local file system (available on all nodes), or any Hadoop-supported file system URI, and return it as an RDD of Strings. Path is the path to the text file on a supported file system, minPartitions is the suggested minimum number of partitions for the resulting RDD. It returns RDD of lines of the text file
sequenceFile [K, V](path: String, keyClass: Class[K], valueClass: Class[V]): RDD[(K, V)]	Get an RDD for a Hadoop SequenceFile with given key and value types. Path is the directory to the input data files, the path can be comma separated paths as a list of inputs, keyClass (resp. valueClass) is the Class of the key (resp. the value) associated with SequenceFileInputFormat. It returns RDD of tuples of key and corresponding value.
parallelize [T](seq: Seq[T], numSlices: Int): RDD[T]	Distribute a local Scala collection to form an RDD. NumSlices is number of partitions to divide the collection into. It returns the RDD representing distributed collection
newAPIHadoopFile [K, V, F <: InputFormat[K, V]](path: String, fClass: Class[F], kClass: Class[K], vClass: Class[V], conf: Configuration = hadoopConfiguration): RDD[(K, V)]	Get an RDD for a given Hadoop file with an arbitrary new API InputFormat and extra configuration options to pass to the input format. Path is the directory to the input data files, the path can be comma separated paths as a list of inputs, fclass is the storage format of the data to be read , kClass (resp. vClass) is the Class of the key (resp. the value) Hadoop configuration. It returns RDD of tuples of key and corresponding value
broadcast [T](value: T): Broadcast[T]	Broadcast a read-only variable to the cluster, returning a org.apache.spark.broadcast.Broadcast object for reading it in distributed functions. The variable will be sent to each cluster only once.
collectionAccumulator [T]: CollectionAccumulator[T]	Create and register a CollectionAccumulator, which starts with empty list and accumulates inputs by adding them into the list.
doubleAccumulator : DoubleAccumulator longAccumulator : LongAccumulator	Create and register a double or long accumulator, which starts with 0 and accumulates inputs by add.
register (acc: AccumulatorV2[_, _]): Unit	Register the given accumulator. Accumulators must be registered before use, or it will throw exception.
stop (): Unit	Shut down the SparkContext.

Simple transformation for RDD[T]	Meaning
map [U](f: (T) =>U): RDD[U]	Return a new distributed dataset formed by passing each element of the source through a function f .
mapValues [U](f: V => U): RDD[(K, U)]	Pass each value in the key-value pair RDD through a map function without changing the keys this also retains the original RDD's partitioning.
filter (f: (T) =>Boolean): RDD[T]	Return a new dataset formed by selecting those elements of the source on which f returns true.
flatMap [U](f: (T) =>TraversableOnce[U]): RDD[U]	Similar to map, but each input item can be mapped to 0 or more output items (so f should return a Seq rather than a single item).
flatMapValues [U](f: V => TraversableOnce[U]): RDD[(K, U)]	Pass each value in the key-value pair RDD through a flatMap function without changing the keys ; this also retains the original RDD's partitioning.

union (other: RDD[T]): RDD[T]	Return a new dataset that contains the union of the elements in the source dataset and the argument.
zip [U](other: RDD[U]): RDD[(T, U)]	Zips this RDD with another one, returning key-value pairs with the first element in each RDD, second element in each RDD, etc. <i>Assumes that the two RDDs have the same number of partitions and the same number of elements in each corresponding partition</i> (e.g. one was made through a map on the other).
<u>zipWithIndex</u> (): RDD[(T, Long)]	Zips this RDD with its element indices. The ordering is first based on the partition index and then the ordering of items within each partition. So the first item in the first partition gets index 0, and the last item in the last partition receives the largest index. This is similar to Scala's zipWithIndex but it uses Long instead of Int as the index type. This method needs to trigger a spark job when this RDD contains more than one partitions.
keyBy [K](f: (T) =>K): RDD[(K, T)]	Creates tuples of the elements in this RDD by applying f
glom (): RDD[Array[T]]	Return an RDD created by coalescing all elements within each partition into an array.

Large transformation for RDD[T]	Meaning
intersection (other: RDD[T]): RDD[T] intersection (other: RDD[T], numPartitions: Int): RDD[T] intersection (other: RDD[T], part: Partitioner): RDD[T]	Return the intersection of this RDD and another one. The output will not contain any duplicate elements, even if the input RDDs did.
distinct (): RDD[T] distinct (numPartitions: Int): RDD[T]	Return a new dataset that contains the distinct elements of the source dataset.
cartesian [U](other: RDD[U]): RDD[(T, U)]	When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements).
repartition (numPartitions: Int): RDD[T]	Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.
subtract (other: RDD[T]): RDD[T] subtract (other: RDD[T], numPartitions: Int): RDD[T] subtract (other: RDD[T], p: Partitioner): RDD[T]	Return an RDD with the elements from <i>this</i> that are not in <i>other</i> .
sortBy [K](f: (T) =>K, ascending: Boolean = true, numPartitions: Int = this.partitions.length): RDD[T]	Return this RDD sorted by the given key function.

Large transformation for RDD[T] and T= (K,V)	Meaning
groupByKey (): RDD[(K, Iterable[V])] groupByKey (numPartitions:Int): RDD[(K, Iterable[V])] groupByKey (p:Partitioner): RDD[(K, Iterable[V])]	When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs.
reduceByKey (func: (V, V) =>V): RDD[(K, V)] reduceByKey (numPartitions:Int, func: (V, V) =>V): RDD[(K, V)] reduceByKey (partitioner: Partitioner, func: (V, V) =>V): RDD[(K, V)]	When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function <i>func</i> , which must be of type (V,V) => V.
sortByKey (ascending: Boolean = true, numPartitions: Int = self.partitions.length) : RDD[(K, V)]	When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean <i>ascending</i> argument.

join [W](other: RDD[(K, W)]): RDD[(K, (V, W))] join [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (V, W))] join [W](other: RDD[(K, W)], p: Partitioner): RDD[(K, (V, W))]	Return an RDD containing all pairs of elements with matching keys in <i>this</i> and <i>other</i> . Each pair of elements will be returned as a (k, (v1, v2)) tuple, where (k, v1) is in <i>this</i> and (k, v2) is in <i>other</i>
leftOuterJoin [W](other: RDD[(K, W)]): RDD[(K, (V, Option[W]))] leftOuterJoin [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (V, Option[W]))] leftOuterJoin [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (V, Option[W]))]	Perform a left outer join of <i>this</i> and <i>other</i> . For each element (k, v) in <i>this</i> , the resulting RDD will either contain all pairs (k, (v, Some(w))) for w in <i>other</i> , or the pair (k, (v, None)) if no elements in <i>other</i> have key k.
rightOuterJoin [W](other: RDD[(K, W)]): RDD[(K, (Option[V], W))] rightOuterJoin [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (Option[V], W))] rightOuterJoin [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (Option[V], W))]	Perform a right outer join of <i>this</i> and <i>other</i> . For each element (k, w) in <i>other</i> , the resulting RDD will either contain all pairs (k, (Some(v), w)) for v in <i>this</i> , or the pair (k, (None, w)) if no elements in <i>this</i> have key k.
fullOuterJoin [W](other: RDD[(K, W)]): RDD[(K, (Option[V], Option[W]))] fullOuterJoin [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, (Option[V], Option[W]))] fullOuterJoin [W](other: RDD[(K, W)], partitioner: Partitioner): RDD[(K, (Option[V], Option[W]))]	Perform a full outer join of <i>this</i> and <i>other</i> . For each element (k, v) in <i>this</i> , the resulting RDD will either contain all pairs (k, (Some(v), Some(w))) for w in <i>other</i> , or the pair (k, (Some(v), None)) if no elements in <i>other</i> have key k. Similarly, for each element (k, w) in <i>other</i> , the resulting RDD will either contain all pairs (k, (Some(v), Some(w))) for v in <i>this</i> , or the pair (k, (None, Some(w))) if no elements in <i>this</i> have key k.
cogroup [W](o: RDD[(K, W)]): RDD[(K, (Iterable[V], Iterable[W]))] cogroup [W](o: RDD[(K, W)], numPart: Int): RDD[(K, (Iterable[V], Iterable[W]))] cogroup [W](o: RDD[(K, W)], p: Partitioner): RDD[(K, (Iterable[V], Iterable[W]))]	When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>)) tuples. This operation is also called groupWith .
subtractByKey [W](other: RDD[(K, W)]): RDD[(K, V)] subtractByKey [W](other: RDD[(K, W)], numPartitions: Int): RDD[(K, V)] subtractByKey [W](other: RDD[(K, W)], p: Partitioner): RDD[(K, V)]	Return an RDD with the pairs from <i>this</i> whose keys are not in <i>other</i>

Action for RDD[T]	Meaning
reduce (f: (T, T) =>T): T	Aggregate the elements of the dataset using a function <i>func</i> (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
collect (): Array[T]	Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
count ():Long	Return the number of elements in the dataset.
first (): T	Return the first element of the dataset (similar to take(1)).
take (num: Int): Array[T]	Return an array with the first <i>n</i> elements of the dataset.
top (num: Int)(implicit ord: Ordering[T]): Array[T]	Returns the top k (largest) elements from this RDD as defined by the specified implicit Ordering[T] and maintains the ordering.
saveAsObjectFile (path: String): Unit	Save this RDD as a SequenceFile of serialized objects.
saveAsTextFile (path: String): Unit	Save this RDD as a text file, using string representations of elements.
foreach (f: (T) =>Unit): Unit	Run a function <i>func</i> on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with external storage systems.

API for StreamingContext	Meaning
new StreamingContext (sparkContext: SparkContext, batchDuration: Duration)	Create a StreamingContext using an existing SparkContext. <i>BatchDuration</i> is the time interval at which streaming data will be divided into batches. Duration can be initialized by <i>Milliseconds(nb)</i> , <i>Seconds(nb)</i> or <i>Minutes(nb)</i>
checkpoint (directory: String)	Set the context to periodically checkpoint the DStream operations for driver fault-tolerance. The directory is HDFS-compatible directory where the checkpoint data will be reliably stored. Note that this must be a fault-tolerant file system like HDFS.
socketTextStream (hostname: String, port: Int, sl: StorageLevel = StorageLevel.MEMORY_AND_DISK_SER_2): ReceiverInputDStream[String]	Creates an input stream from TCP source hostname:port. Data is received using a TCP socket and the receive bytes is interpreted as UTF8 encoded <code>\n</code> delimited lines. The <i>hostname</i> parameter is the host to connect to for receiving data, <i>port</i> is the port to connect to for receiving data, <i>sl</i> is the Storage level to use for storing the received objects.
socketStream [T: ClassTag](hostname: String, port: Int, converter: (InputStream) => Iterator[T], sl: StorageLevel): ReceiverInputDStream[T]	Create an input stream from network source hostname:port, where data is received as serialized blocks (serialized using the Spark's serializer) that can be directly pushed into the block manager without deserializing them. This is the most efficient way to receive data. The <i>hostname</i> parameter is the host to connect to for receiving data, <i>port</i> is the port to connect to for receiving data, <i>sl</i> is the Storage level to use for storing the received objects. T is the type of the objects in the received blocks.
textFileStream (directory: String): DStream[String]	Create an input stream that monitors a Hadoop-compatible filesystem for new files and reads them as text files (using key as LongWritable, value as Text and input format as TextInputFormat). Files must be written to the monitored directory by "moving" them from another location within the same file system. File names starting with . are ignored.
start () : Unit	Start the execution of the streams. Throws IllegalStateException if the StreamingContext is already stopped.
awaitTermination ()	Wait for the execution to stop. Any exceptions that occurs during the execution will be thrown in this thread.
awaitTerminationOrTimeout (timeout: Long): Boolean	Wait for the execution to stop. Any exceptions that occurs during the execution will be thrown in this thread. The <i>timeout</i> parameter is the time to wait in milliseconds. Returns true if it's stopped; or throw the reported error during the execution; or false if the waiting time elapsed before returning from the method.
stop (stopSparkContext: Boolean = conf.getBoolean("spark.streaming.stopSparkContextByDefault", true)): Unit	Stop the execution of the streams immediately (does not wait for all received data to be processed). By default, if <i>stopSparkContext</i> is not specified, the underlying SparkContext will also be stopped. This implicit behavior can be configured using the SparkConf configuration <i>spark.streaming.stopSparkContextByDefault</i> . If <i>stopSparkContext</i> is true, it stops the associated SparkContext. The underlying SparkContext will be stopped regardless of whether this StreamingContext has been started.

Transformation for DStream[T]	Meaning
filter (filterFunc: (T) => Boolean): DStream[T]	Return a new DStream containing only the elements that satisfy a predicate.
map [U](mapFunc: (T) => U): DStream[U]	Return a new DStream by applying a function to all elements of this DStream
flatMap [U](flatMapFunc: (T) => TraversableOnce[U]): DStream[U]	Return a new DStream by applying a function to all elements of this DStream, and then flattening the results
glom (): DStream[Array[T]]	Return a new DStream in which each RDD is generated by applying glom() to each RDD of this DStream. Applying glom() to an RDD coalesces all elements within each partition into an array.
union (that: Dstream[T]): DStream[T]	Return a new DStream by unifying data of another DStream with this DStream.
repartition (numPartitions: Int): DStream[T]	Return a new DStream with an increased or decreased level of parallelism. Each RDD in the returned DStream has exactly numPartitions partitions.
count (): DStream[Long]	Return a new DStream in which each RDD has a single element generated by counting each RDD of this DStream.
countByValue (numPartitions: Int = ssc.sc.defaultParallelism)(implicit ord: Ordering[T] = null): DStream[(T, Long)]	Return a new DStream in which each RDD contains the counts of each distinct value in each RDD of this DStream. Hash partitioning is used to generate the RDDs with numPartitions partitions (Spark's default number of partitions if numPartitions not specified).
countByValueAndWindow (windowDuration: Duration, slideDuration: Duration, numPartitions: Int = ssc.sc.defaultParallelism)(implicit ord: Ordering[T] = null): DStream[(T, Long)]	Return a new DStream in which each RDD contains the count of distinct elements in RDDs in a sliding window over this DStream. Hash partitioning is used to generate the RDDs with numPartitions partitions (Spark's default number of partitions if numPartitions not specified).
transform [U](transformFunc: (RDD[T], Time) => RDD[U])(implicit arg0: ClassTag[U]): DStream[U]	Return a new DStream in which each RDD is generated by applying a function on each RDD of 'this' DStream.
reduce (reduceFunc: (T, T) => T): DStream[T]	Return a new DStream in which each RDD has a single element generated by reducing each RDD of this DStream.
foreachRDD (foreachFunc: (RDD[T]) => Unit): Unit	Apply a function to each RDD in this DStream. This is an output operator, so 'this' DStream will be registered as an output stream and therefore materialized.
countByWindow (windowDuration: Duration, slideDuration: Duration): DStream[Long]	Return a new DStream in which each RDD has a single element generated by counting the number of elements in a sliding window over this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions
reduceByWindow (reduceFunc: (T, T) => T, windowDuration: Duration, slideDuration: Duration): DStream[T]	Return a new DStream in which each RDD has a single element generated by reducing all elements in a sliding window over this DStream.
window (windowDuration: Duration, slideDuration: Duration): DStream[T]	Return a new DStream in which each RDD contains all the elements in seen in a sliding window of time over this DStream.

Transformation for DStream[T], T= (K,V)	Meaning
combineByKey [C](createCombiner: (V) => C, mergeValue: (C, V) => C, mergeCombiner: (C, C) => C, partitioner: Partitioner, mapSideCombine: Boolean = true): DStream[(K, C)]	Combine elements of each key in DStream's RDDs using custom functions. This is similar to the combineByKey for RDDs. Please refer to combineByKey in org.apache.spark.rdd.PairRDDFunctions in the Spark core documentation for more information.
reduceByKey (reduceFunc: (V, V) => V): DStream[(K, V)]	Return a new DStream by applying reduceByKey to each RDD. The values for each key are merged using the associative and commutative reduce function. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
mapValues [U](mapValuesFunc: (V) => U): DStream[(K, U)]	Return a new DStream by applying a map function to the value of each key-value pairs in 'this' DStream without changing the key
flatMapValues [U](flatMapValuesFunc: (V) => TraversableOnce[U]): DStream[(K, U)]	Return a new DStream by applying a flatmap function to the value of each key-value pairs in 'this' DStream without changing the key.
join [W](other: DStream[(K, W)]): DStream[(K, (V, W))]	Return a new DStream by applying 'join' between RDDs of this DStream and other DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
leftOuterJoin [W](other: DStream[(K, W)]): DStream[(K, (V, Option[W]))]	Return a new DStream by applying 'left outer join' between RDDs of this DStream and other DStream.
fullOuterJoin [W](other: DStream[(K, W)]): DStream[(K, (Option[V], Option[W]))]	Return a new DStream by applying 'full outer join' between RDDs of this DStream and other DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
rightOuterJoin [W](other: DStream[(K, W)]): DStream[(K, (Option[V], W))]	Return a new DStream by applying 'right outer join' between RDDs of this DStream and other DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
cogroup [W](other: DStream[(K, W)])(implicit arg0: ClassTag[W]): DStream[(K, (Iterable[V], Iterable[W]))]	Return a new DStream by applying 'cogroup' between RDDs of this DStream and other DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
groupByKey (): DStream[(K, Iterable[V])]	Return a new DStream by applying groupByKey to each RDD. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
groupByKeyAndWindow (windowDuration: Duration): DStream[(K, Iterable[V])]	Return a new DStream by applying groupByKey over a sliding window. This is similar to DStream.groupByKey() but applies it over a sliding window. The new DStream generates RDDs with the same interval as this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
reduceByKeyAndWindow (reduceFunc: (V, V) => V, windowDuration: Duration): DStream[(K, V)]	Return a new DStream by applying reduceByKey over a sliding window on this DStream. Similar to DStream.reduceByKey(), but applies it over a sliding window. The new DStream generates RDDs with the same interval as this DStream. Hash partitioning is used to generate the RDDs with Spark's default number of partitions.

reduceByKeyAndWindow (reduceFunc: (V, V) => V, invReduceFunc: (V, V) => V, windowDuration: Duration, slideDuration: Duration = self.slideDuration, numPartitions: Int = ssc.sc.defaultParallelism, filterFunc: ((K, V)) => Boolean = null): DStream[(K, V)]	Return a new DStream by applying incremental `reduceByKey` over a sliding window. The reduced value of over a new window is calculated using the old window's reduced value : 1. reduce the new values that entered the window (e.g., adding new counts) 2. "inverse reduce" the old values that left the window (e.g., subtracting old counts) This is more efficient than reduceByKeyAndWindow without "inverse reduce" function. However, it is applicable to only "invertible reduce functions". Hash partitioning is used to generate the RDDs with Spark's default number of partitions.
updateStateByKey [S](updateFunc: (Seq[V], Option[S]) => Option[S]):DStream[(K,S)]	Return a new "state" DStream where the state for each key is updated by applying the given function on the previous state of the key and the new values of each key. In every batch the updateFunc will be called for each state even if there are no new values. Hash partitioning is used to generate the RDDs with Spark's default number of partitions. If updateFunc function returns None, then corresponding state key-value pair will be eliminated.

Output Operation for DStream[T]	Meaning
foreachRDD (foreachFunc: (RDD[T]) => Unit): Unit	Apply a function to each RDD in this DStream. This is an output operator, so 'this' DStream will be registered as an output stream and therefore materialized.
print (num: Int): Unit	Print the first num elements of each RDD generated in this DStream. This is an output operator, so this DStream will be registered as an output stream and there materialized.
print (): Unit	Print the first ten elements of each RDD generated in this DStream. This is an output operator, so this DStream will be registered as an output stream and there materialized.
saveAsObjectFiles (prefix: String, suffix: String = ""): Unit	Save each RDD in this DStream as a Sequence file of serialized objects. The file name at each batch interval is generated based on <code>prefix</code> and <code>suffix</code> : "prefix-TIME_IN_MS.suffix".
saveAsTextFiles (prefix: String, suffix: String = ""): Unit	Save each RDD in this DStream as at text file, using string representation of elements.
saveAsNewAPIHadoopFiles [F <: OutputFormat[K, V]](prefix: String, suffix: String): Unit	Save each RDD in this DStream as a Hadoop file. The file name at each batch interval is generated based on <code>prefix</code> and <code>suffix</code> : "prefix-TIME_IN_MS.suffix".